

Database Management System (DBMS): A DBMS is software that manages database structure and controls data access, acting as an intermediary between users and the database.

Role of a DBMS: Enables data sharing and integration. Provides an integrated view of data. Translates application requests. Hides database complexity from users

Advantages: Better data integration and consistency. Improved data sharing, security, access, and decision-making. Higher data quality and user productivity

Functions: Data dictionary management: Stores data definitions and relationships. Data storage management: Ensures efficient performance. Data transformation and presentation: Formats data as required. Security management: Enforces data privacy. Multiuser access control: Maintains integrity during concurrent access. Backup and recovery: Restores data after failures. Data integrity management: Reduces redundancy and ensures consistency. Database access languages: SQL used for querying and managing data.

Hierarchical Model (Tree Structure):

- Structure:** Each child record has exactly one parent. Models 1:M relationships well.
- Navigation:** Requires knowing the physical access path from the root.
- Cons:** Structurally dependent; difficult to represent M:N relationships.

Network Model (Graph Structure):

- Structure:** A record can have multiple parents, allowing it to model M:N relationships directly.
- Navigation:** Still navigational (path-dependent), but more flexible than hierarchical.
- Cons:** Complex to implement and manage; applications are still tied to the structure.

Key Distinction: Both are **navigational**, requiring programs to know the data paths. The relational model is **declarative**, where users specify *what* they want, not *how* to get it.

Primary Key (PK) "Pointing": The proper database terminology for a primary key "pointing" to all the non-prime attributes (determining their values) is Determination or functional dependency. The key is used to determine other (non-prime) values of the row. PK-Related Issue (Single Table): The issue that can occur in a single table related to a PK is called entity integrity violation. It happens when entire rows become identical because of null or replicate (identical) Primary Keys. PK-Related Issue (Two Related Tables): The issue that can occur in two related tables is a violation of Referential Integrity. This happens when foreign keys in one table refer to an absent or deleted row in the other table.

Problems with File System Data Processing: Lengthy development times and extensive programming. Difficulty of getting quick answers. Complex system administration. Lack of security and limited data sharing.

Structural dependence: Access to a file depends on its own structure, meaning programs must be modified if the file structure changes. Structural independence is the goal (changing file structure without affecting applications).

Data dependence: Data access changes when data storage characteristics change. Data independence is the goal (changing storage characteristics without affecting application access).

Data Redundancy: Unnecessarily storing the same data in different places, which creates "islands of information" and increases the probability of having different versions of the same data.

Implications of Data Redundancy: Poor data security, data inconsistency, increased data-entry errors, and data anomalies (update, insertion, and deletion anomalies) that occur when required changes in redundant data are not made successfully.

Object-Oriented (O-O) and Extended Relational (O/R DBMS): O-O supports complex objects, inheritance, and encapsulation, but is harder to query. O/R DBMS is a compromise, often using an O-O front-end over a relational architecture (ORM).

Join: Allows information to be intelligently combined from two or more tables.

CROSS JOIN: Returns the Cartesian product of the two tables.

INNER JOIN: Returns only the rows that meet the join condition.

Natural Join: Returns rows with matching values in matching columns which must have the same names and similar data types.

OUTER JOIN: Returns rows with matching values and includes all rows from one or both tables with unmatched values.

LEFT JOIN/LEFT OUTER JOIN: Returns matching rows and includes all rows from the left table with unmatched values.

RIGHT JOIN/RIGHT OUTER JOIN: Returns matching rows and includes all rows from the right table with unmatched values.

FULL JOIN/FULL OUTER JOIN: Returns matching rows and includes all rows from both tables with unmatched values.

Recursive Joins (Self-Join): A table is joined to itself using aliases to differentiate the table's role.

Data Models are simple representations of complex real-world data structures that are useful for supporting a specific problem domain. Importance of **Business Rules** are brief, precise, and unambiguous descriptions of a policy, procedure, or principle. They are a source of entities and relationships.

Translating Business Rules: nouns become entities, verbs become relationships. Documentation standardizes the company's data view and supports clear communication.

Edgar F. Codd's 1970 Relational Model introduced data abstraction by storing all data in tables linked by keys instead of physical pointers, ensuring data independence. Based on relational algebra, it enabled powerful querying (via SQL) and became the foundation of modern databases.

The Relational Model represents data logically using tables, where rows (tuples) are entity occurrences and columns (attributes) follow a defined domain. Row and column order doesn't matter, and each table requires a unique key to identify rows.

Weak Entity: An entity that is both existence-dependent and has a primary key that is partially or totally derived from the parent entity in the relationship (a strong/identifying relationship).

Strong (Identifying) Relationship: The primary key of the related (dependent) entity contains a primary key component of the parent entity.

Weak (Non-identifying) Relationship: The primary key of the related entity does not contain a primary key component of the parent entity.

Good table/ relational DB: Use simple, numeric primary keys (avoid meaningful or "intelligent" keys). Normalize data to eliminate redundancy and ensure integrity. Apply clear, consistent naming conventions for tables and attributes. Index non-PK columns often used in searches or joins. Use bridge (associative) entities to manage many-to-many relationships efficiently.

Correlated Subquery: A subquery becomes correlated because it **references one or more columns from the outer ('main') query**. As a result, the subquery must be **evaluated once per row of the outer query**. This can be highly inefficient compared to a single-time join.

Closure ensures that the result of an operation stays within the same domain or type. For relational databases, closure means that operations performed on tables (relations), such as SELECT, UNION, JOIN, etc., must result in a table as output. Closure is considered a Big Functional Deal (BFD) for table operations because it is essential for chaining operations. Since the output is always a table, the result can immediately serve as the input for the next operation without "breaking the system".

Applications evolved from monolithic, centralized systems to modern microservices by first using middleware and connectors like ODBC, JDBC, and COM/CORBA, then adopting web-based scripts (cgi-bin) and client-side support (ActiveX, Node) to enable multi-database access.

CROW'S FOOT SYMBOLS	CARDINALITY	COMMENT
	{0,N}	Zero or many; the "many" side is optional.
	{1,N}	One or many; the "many" side is mandatory.
	{1,1}	One and only one; the "1" side is mandatory.
	{0,1}	Zero or one; the "1" side is optional.

TYPE	DISJOINT CONSTRAINT	OVERLAPPING CONSTRAINT
Partial 	Supertype has optional subtypes. Subtype discriminator can be null. Subtype sets are unique.	Supertype has optional subtypes. Subtype discriminators can be null. Subtype sets are not unique.
Total 	Every supertype occurrence is a member of only one subtype. Subtype discriminator cannot be null. Subtype sets are unique.	Every supertype occurrence is a member of at least one subtype. Subtype discriminators cannot be null. Subtype sets are not unique.

PL/SQL is a procedural extension to SQL that allows for conditional or looping operations by isolating critical code. It allows for the creation and storage of procedural code and SQL statements within the database. End users can use PL/SQL to create: **Anonymous PL/SQL blocks** for one-time execution.

Triggers: Procedural SQL code automatically invoked by the RDBMS when a given data manipulation event (INSERT, UPDATE, or DELETE) occurs. Can be **BEFORE** or **AFTER** the triggering statement. Can be **statement-level** (default, executes once) or **row-level** (requires **FOR EACH ROW**, executes once for each row affected). **Stored Procedures:** Named collection of procedural and SQL statements for repeated calling, without return values. **Stored Functions:** Named group of procedural and SQL statements that *returns a value* (indicated by a **RETURN** statement) for repeated calling.

Normalization is a design step involving the evaluation and correction of table structures to minimize data redundancies and reduce data anomalies. It assigns attributes to tables based on determination. The primary goals of normalization are to reduce redundancies and anomalies. It is best performed early in the overall design process. Dependency Diagram: This diagram helps visualize and identify partial and transitive dependencies, which is crucial for the normalization process. First Normal Form (1NF): Tabular format, no repeating groups, all attributes are dependent on the primary key, all key attributes are defined. The table may still contain partial and transitive dependencies. 2NF: Achieved by eliminating partial dependencies. 3NF: Achieved by eliminating transitive dependencies.	The Problem with "Too Much" Normalization: Excessive table normalization leads to an increase in query latency/cost and a decrease in efficiency due to the high number of JOINS that must occur to retrieve data spread across many small tables, which reduces system speed. Remedy for Over-Normalization: The remedy is table denormalization, a process by which a table is changed from a higher normal form to a lower normal form, typically to increase processing speed and simplify queries. It results in increased performance and greater data redundancy. Denormalization may be necessary because decomposing tables to meet normalization requirements expands the number of tables. Normalization's Effect on Database Design: Data Integrity: Goes up because data is kept in just a single location and referenced elsewhere. Data Updates: Becomes easy and fast since fewer parts of the database (or fewer indexes) need updating. Concurrency: Also increases, as locking is restricted to only the tables/rows where contention occurs. A deadlock (also known as a deadly embrace) occurs when two transactions wait indefinitely for each other to unlock data. For example, Transaction 1 locks item A and waits for item B, which is locked by Transaction 2. Transaction 2, meanwhile, waits for item A, which is locked by Transaction 1. Neither can proceed without Transaction Manager intervention or through a prevention scheme. Deadlocks can be controlled through: Deadlock Prevention: The transaction is aborted and rolled back if a deadlock is possible, then rescheduled. Deadlock Detection: The DBMS periodically tests for deadlocks, and if one is found, a "victim" transaction is aborted (rolled back and restarted) while the other continues. Deadlock Avoidance: The transaction must obtain all the locks it needs before execution, requiring serial lock assignment. Deadlock with 2PL: 2PL cannot entirely prevent deadlocks. A deadlock is a lock starvation problem, a circular wait where two or more transactions wait indefinitely for each other to release a lock.	ACID Properties: The desirable properties of a transaction are captured by the ACID acronym: Atomicity: All operations of a transaction must be completed; otherwise, the transaction is aborted. Consistency: Ensures the permanence of the database's consistent state. Isolation: Data used during a transaction cannot be used by a second transaction until the first is completed. Durability: Ensures that once transactions are committed, they cannot be undone or lost. Serializability (S in the full acronym): Ensures that the schedule for the concurrent execution of several transactions will yield consistent results (the same as serial execution). Problems in Concurrency Control When concurrent transactions are not properly handled, three kinds of errors can occur: Lost Update: Occurs when the same data element is updated by two concurrent transactions, and one of the updates is lost. Uncommitted Data (Dirty Read): Occurs when one concurrent transaction reads uncommitted data written by a first transaction, which is later rolled back. Inconsistent Retrievals: Occurs when a transaction accesses data before and after one or more other transactions finish working with that data. This can also be called a 'dirty read' Deadlock prevention using time stamping includes schemes like: Wait/Die Scheme: An older (requesting) transaction makes a younger (owning) transaction die (roll back and reschedule). A younger (requesting) transaction must wait for an older (owning) transaction to finish. Wound/Wait Scheme: An older (requesting) transaction wounds (forces roll back) a younger (owning) transaction. A younger (requesting) transaction must wait for an older (owning) transaction. Optimistic Concurrency Control (OCC) The optimistic approach is a 'fix-it-later' method that is indifferent to deadlocks and involves three phases: Read: The transaction reads the database, executes computations, and makes updates to a private copy of the data. Validation: The private changes are validated to ensure they will not affect the integrity and consistency of the database. Write: Changes are permanently applied to the database. Distributed Concurrency Control: Two-Phase Commit (2PC) A protocol that guarantees if any part of a transaction cannot be committed, all changes made at other sites will be undone. 2PC is a distributed commit protocol used in distributed databases to ensure that a single distributed transaction is atomic (all nodes commit or all abort) across an entire collection of nodes. Phases: Preparation (Voting): The coordinator polls all participant nodes and asks them if they are ready to commit. Commit/Abort (Execution): If all nodes vote "yes," the coordinator instructs them to commit. Otherwise, it instructs them to abort. Failure Modes: 2PC can fail if the coordinator node fails or if network partitioning occurs during the second (commit) phase, meaning a commit could fail after commencing. Levels of Distribution: SPSD (Single-Site Processing, Single-Site Data): A centralized host computer processes data stored on its local disk. MPSP: Multiple computers share a single data repository, often in a client/server architecture. MPMD: A fully distributed system with multiple data processors and transaction processors at various sites. Can be homogeneous (same DBMS) or heterogeneous (different DBMSs).
Functional Dependencies: A functional dependence exists when an attribute B is fully functionally dependent on attribute A if each value of A determines one and only one value of B. Types of Functional Dependencies: Partial dependency: A functional dependence where the determinant is only part of the primary key. This means a part of an existing Primary Key (PK) is acting like a PK on its own. Transitive dependency: An attribute functionally depends on another nonkey attribute. This means a non-PK (regular attribute) is acting like a PK.		
Two-Phase Locking (2PL) defines how transactions acquire and relinquish locks, and it guarantees serializability but does not prevent deadlocks. The phases are: Growing phase: The transaction acquires all required locks without unlocking any data. Shrinking phase: The transaction releases all locks and cannot obtain any new lock. The point where the last lock is acquired is called the locked point. Cascading Aborts (Problem with Releasing): This is an issue in the most basic form of 2PL. If a transaction T1 releases an Exclusive (Write) Lock before committing, and another transaction T2 acquires that lock and updates the data item, the commit of T2 must also be rolled back. This domino effect leads to wasted work and your database recovering slowly		
Data Fragmentation: Breaking a single database object into multiple segments. Horizontal: Divides a table into subsets of rows (tuples). Vertical: Divides a table into subsets of columns (attributes). Mixed: A combination of both. Data Replication: Storing copies of data fragments at multiple sites. This can be a fully replicated, partially replicated, or unreplicated database. Data Allocation: Deciding where to locate fragments and replicas. BASE: Used in NoSQL Basically Available: The system ensures high availability. It sacrifices consistency to do so. Soft state: This principle acknowledges that the data's state may change over time as updates propagate through the system. Eventually consistent: Data changes are not immediate. Instead, they spread slowly through the system until all replicas are eventually consistent and identical. The CAP Theorem In a distributed system, can only achieve two: Consistency: All nodes see the same data at the same time. Availability: Every request receives a response, without guarantee that it contains the most recent version. Partition Tolerance: The system continues to operate despite network failures. Partition Tolerance (P) is a given (unavoidable). The real trade-off is between Consistency (C) and Availability (A) when a partition failure occurs. Prioritize Consistency (C): Refuse a write request during a partition to prevent inconsistency, thereby prioritizing consistency at the expense of lowering availability. Prioritize Availability (A): Accept a write request during a partition to remain operational, thereby prioritizing availability at the expense of potential, eventual inconsistency (BASE principle).		